

# Краткие TeX-заготовки ответов к экзамену по RL

по программе экзамена и лекциям курса

## Содержание

|    |                                                                        |    |
|----|------------------------------------------------------------------------|----|
| 1  | Кросс-энтропийный метод: общий вид, оптимизация и RL                   | 2  |
| 2  | Уравнения Беллмана. Policy Iteration и Value Iteration                 | 5  |
| 3  | Табличные методы: Монте-Карло и Q-learning                             | 8  |
| 4  | DQN и модификации: Double DQN, PER, dueling, noisy, multi-step, memory | 11 |
| 5  | Distributional RL: QR-DQN и Implicit Quantile Networks                 | 14 |
| 6  | Внутренняя мотивация: RND и ICM                                        | 17 |
| 7  | Policy Gradient. REINFORCE и модификации                               | 20 |
| 8  | Actor-Critic. A2C и GAE                                                | 23 |
| 9  | TRPO: Trust Region Policy Optimization                                 | 26 |
| 10 | Bias-variance, GAE и PPO                                               | 28 |
| 11 | DDPG и TD3: off-policy actor-critic для непрерывного управления        | 31 |
| 12 | Entropy RL и Soft Actor-Critic                                         | 34 |
| 13 | Имитационное обучение, offline RL, Guided Cost Learning и GAIL         | 37 |
| 14 | Многорукые бандиты: UCB и Thompson Sampling                            | 40 |
| 15 | Monte Carlo Tree Search                                                | 43 |
| 16 | LQR, iLQR и общая схема Model-based RL                                 | 46 |

# 1 Кросс-энтропийный метод: общий вид, оптимизация и RL

## Идея

Кросс-энтропийный метод (Cross-Entropy Method, CEM) можно понимать как метод поиска хорошего распределения над решениями. Мы не оптимизируем одну точку напрямую, а задаем параметрическое семейство распределений  $q_\theta(x)$  над кандидатами  $x$ , сэмплируем кандидатов, отбираем элиту и подгоняем распределение так, чтобы оно с большей вероятностью порождало элитные решения.

Общая задача оптимизации:

$$x^* \in \arg \max_{x \in \mathcal{X}} S(x),$$

где  $S(x)$  - целевая функция. Вместо прямого поиска  $x^*$  вводим распределение  $q_\theta(x)$  и итеративно сдвигаем его массу в сторону областей, где  $S(x)$  велико.

## Редкое событие и условное распределение на успехе

Исторически CEM связан с оценкой вероятности редких событий. Пусть  $X \sim p(x)$  и нас интересует событие

$$S(X) \geq \gamma.$$

Идеальное распределение для importance sampling - это исходное распределение, условленное на успех:

$$p^*(x) = p(x \mid S(x) \geq \gamma) = \frac{p(x) \mathbf{1}\{S(x) \geq \gamma\}}{\mathbb{P}_p(S(X) \geq \gamma)}.$$

Здесь  $\mathbf{1}\{\cdot\}$  - индикатор: он равен 1, если условие выполнено, и 0 иначе. Такое распределение идеально, потому что оно сэмплирует только успешные объекты. На практике  $p^*$  неизвестно, поэтому его приближают распределением  $q_\theta$  из удобного семейства.

Параметры выбирают через минимизацию KL-дивергенции:

$$\theta^* = \arg \min_{\theta} \text{KL}(p^* \| q_\theta).$$

Раскрывая KL,

$$\text{KL}(p^* \| q_\theta) = \mathbb{E}_{p^*} \left[ \log \frac{p^*(X)}{q_\theta(X)} \right] = \mathbb{E}_{p^*} [\log p^*(X)] - \mathbb{E}_{p^*} [\log q_\theta(X)].$$

Первое слагаемое не зависит от  $\theta$ , поэтому задача эквивалентна максимизации правдоподобия элитных сэмплов:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{p^*} \log q_\theta(X).$$

Отсюда возникает логарифм: это обычная MLE-задача для распределения, которое должно порождать успешные примеры.

## Практический алгоритм CEM

Поскольку  $p^*$  неизвестно, его заменяют эмпирическим распределением на элитных сэмплах.

На итерации  $k$ :

1. Имеем текущее распределение  $q_{\theta_k}(x)$ .
2. Сэмплируем кандидатов:

$$x_1, \dots, x_N \sim q_{\theta_k}(x).$$

3. Считаем оценки качества  $S(x_i)$ .
4. Выбираем порог элиты  $\gamma_k$ , например  $(1 - \rho)$ -квантиль по значениям  $S(x_i)$ , где  $\rho \in (0, 1)$  - доля элитных объектов.
5. Формируем элитное множество:

$$\mathcal{E}_k = \{x_i : S(x_i) \geq \gamma_k\}.$$

6. Обновляем параметры распределения по MLE:

$$\theta_{k+1} = \arg \max_{\theta} \sum_{x_i \in \mathcal{E}_k} \log q_{\theta}(x_i).$$

Если  $q_{\theta}$  - нормальное распределение  $\mathcal{N}(\mu, \Sigma)$ , то обновление имеет простой вид:

$$\mu_{k+1} = \frac{1}{|\mathcal{E}_k|} \sum_{x_i \in \mathcal{E}_k} x_i, \quad \Sigma_{k+1} = \frac{1}{|\mathcal{E}_k|} \sum_{x_i \in \mathcal{E}_k} (x_i - \mu_{k+1})(x_i - \mu_{k+1})^{\top}.$$

Часто используют сглаживание:

$$\theta_{k+1} \leftarrow \alpha \theta_{k+1}^{\text{MLE}} + (1 - \alpha) \theta_k,$$

чтобы распределение не схлопнулось слишком быстро.

## СЕМ как метод оптимизации

В детерминированной оптимизации событие успеха задают через высокий уровень целевой функции. Тогда СЕМ можно записать как итеративное приближение распределения

$$q_{k+1}(x) \approx q_k(x \mid S(x) \geq \gamma_k).$$

Это интуитивно: берем текущее распределение, смотрим, какие сэмплы оказались лучшими, и обучаем новое распределение быть похожим на распределение лучших сэмплов.

В отличие от градиентных методов, СЕМ:

- не требует дифференцируемости  $S(x)$ ;
- естественно работает с дискретными, непрерывными и смешанными переменными;
- может быть устойчив к локальной негладкости;
- но требует много сэмплов и страдает в больших размерностях.

## СЕМ в обучении с подкреплением

В RL объектом оптимизации может быть не одно действие, а политика или целая траектория. Пусть политика параметризована вектором  $\theta$ , а качество политики:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} R(\tau), \quad R(\tau) = \sum_{t=0}^T \gamma^t r_t.$$

Тогда СЕМ можно применять как black-box оптимизацию параметров политики:

1. Задать распределение над параметрами политики  $q_{\psi}(\theta)$ .
2. Насэмплировать  $\theta_1, \dots, \theta_N \sim q_{\psi}$ .
3. Для каждой политики  $\pi_{\theta_i}$  сыграть один или несколько эпизодов и оценить  $J(\theta_i)$ .
4. Выбрать элитные параметры с наибольшей средней наградой.
5. Обновить  $q_{\psi}$  по MLE на элитных  $\theta_i$ .

Если политика линейная или небольшая нейросеть, это может быть вполне рабочим методом. В таком варианте СЕМ не использует градиенты по политике и не требует знания динамики среды.

## СЕМ для поиска действий или планов

Другой вариант - применять СЕМ не к параметрам политики, а к последовательности действий:

$$a_{0:H-1} = (a_0, a_1, \dots, a_{H-1}).$$

Если есть симулятор или модель динамики, можно решать задачу планирования:

$$a_{0:H-1}^* \in \arg \max_{a_{0:H-1}} \mathbb{E} \left[ \sum_{t=0}^{H-1} \gamma^t r(s_t, a_t) \right].$$

СЕМ задает распределение над планами, например

$$q_{\theta}(a_{0:H-1}) = \prod_{t=0}^{H-1} \mathcal{N}(a_t \mid \mu_t, \Sigma_t),$$

сэмплирует планы, прогоняет их через модель, выбирает элиту и обновляет  $\mu_t, \Sigma_t$ . На реальном шаге обычно исполняют только первое действие  $a_0$ , затем получают новое состояние и заново перепланируют. Это вариант MPC (model predictive control).

### Связь с MLE и названием метода

Название “cross-entropy” связано с тем, что минимизация

$$\text{KL}(p^* \parallel q_{\theta})$$

эквивалентна минимизации кросс-энтропии между идеальным распределением успешных объектов и модельным распределением:

$$H(p^*, q_{\theta}) = -\mathbb{E}_{p^*} \log q_{\theta}(X).$$

А практически это превращается в MLE по элитным сэмплам. Поэтому СЕМ можно запомнить так: “сэмплируем - отбираем лучших - обучаем распределение на лучших”.

### Что обязательно сказать на экзамене

Кросс-энтропийный метод - это не один конкретный RL-алгоритм, а общий принцип. Он строит параметрическое распределение над решениями и обновляет его через MLE по элитным сэмплам. В RL решениями могут быть параметры политики, траектории или планы действий. Плюс метода - отсутствие требований к дифференцируемости; минус - высокая сэмпловая стоимость и плохая масштабируемость без дополнительных трюков.

## 2 Уравнения Беллмана. Policy Iteration и Value Iteration

### MDP и функции ценности

Марковский процесс принятия решений задается кортежем

$$M = (\mathcal{S}, \mathcal{A}, p, r, \gamma),$$

где  $\mathcal{S}$  - состояния,  $\mathcal{A}$  - действия,  $p(s' | s, a)$  - вероятность перехода,  $r(s, a)$  - средняя награда,  $\gamma \in (0, 1]$  - дисконтирование. Политика  $\pi(a | s)$  задает распределение действий в состоянии  $s$ .

Для фиксированной политики  $\pi$  вводятся функции ценности:

$$V^\pi(s) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right],$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right].$$

$V^\pi(s)$  - насколько хорошо начать из состояния  $s$  и дальше действовать по  $\pi$ .  $Q^\pi(s, a)$  - насколько хорошо сначала сделать  $a$  из  $s$ , а затем действовать по  $\pi$ .

### Уравнения Беллмана для фиксированной политики

Связь между  $V^\pi$  и  $Q^\pi$ :

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) Q^\pi(s, a),$$

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V^\pi(s').$$

Отсюда уравнение Беллмана для  $V^\pi$ :

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left[ r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V^\pi(s') \right].$$

В матричной форме:

$$V^\pi = r^\pi + \gamma P^\pi V^\pi,$$

где

$$r^\pi(s) = \sum_a \pi(a | s) r(s, a), \quad P^\pi(s' | s) = \sum_a \pi(a | s) p(s' | s, a).$$

Если  $\gamma < 1$ , то

$$V^\pi = (I - \gamma P^\pi)^{-1} r^\pi.$$

На практике часто используют итерации Беллмана:

$$V_{k+1}(s) = \sum_a \pi(a | s) \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V_k(s') \right].$$

Оператор Беллмана для фиксированной политики является сжимающим с коэффициентом  $\gamma$  в  $\text{sup-норме}$ , поэтому итерации сходятся к единственному  $V^\pi$ .

## Оптимальные функции ценности

Оптимальная ценность:

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a).$$

Оптимальные уравнения Беллмана:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V^*(s') \right],$$
$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) \max_{a' \in \mathcal{A}} Q^*(s', a').$$

Оптимальная детерминированная политика получается жадно:

$$\pi^*(s) \in \arg \max_{a \in \mathcal{A}} Q^*(s, a) = \arg \max_{a \in \mathcal{A}} \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V^*(s') \right].$$

Главная идея: если мы знаем  $Q^*$ , то задача решена, потому что в каждом состоянии надо выбрать действие с максимальным  $Q^*(s, a)$ .

## Policy Evaluation

Policy Evaluation - оценивание  $V^{\pi}$  для фиксированной политики  $\pi$ . Вход: MDP и политика. Выход:  $V^{\pi}$ .

Итерационный вариант:

$$V_{k+1} = T^{\pi} V_k,$$

где

$$(T^{\pi} V)(s) = \sum_a \pi(a | s) \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V(s') \right].$$

Остановка: например,

$$\|V_{k+1} - V_k\|_{\infty} < \varepsilon.$$

## Policy Improvement

Policy Improvement - улучшение политики по текущей оценке  $V^{\pi}$  или  $Q^{\pi}$ :

$$\pi_{\text{new}}(s) \in \arg \max_a Q^{\pi}(s, a).$$

Если

$$Q^{\pi}(s, \pi_{\text{new}}(s)) \geq V^{\pi}(s) \quad \forall s,$$

то новая политика не хуже старой:

$$V^{\pi_{\text{new}}}(s) \geq V^{\pi}(s).$$

Это утверждение называется policy improvement theorem.

## Policy Iteration

Policy Iteration чередует оценивание политики и ее улучшение.

Алгоритм:

1. Инициализировать произвольную политику  $\pi_0$ .
2. Для  $k = 0, 1, 2, \dots$ :
  - (a) Policy Evaluation: найти  $V^{\pi_k}$  из

$$V^{\pi_k} = T^{\pi_k} V^{\pi_k}.$$

(b) Policy Improvement:

$$\pi_{k+1}(s) \in \arg \max_a \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V^{\pi_k}(s') \right].$$

(c) Если  $\pi_{k+1} = \pi_k$ , остановиться: политика оптимальна.

В конечном MDP с дисконтированием метод сходится за конечное число улучшений, потому что детерминированных политик конечное число, а каждое нетривиальное улучшение делает политику лучше.

## Value Iteration

Value Iteration объединяет оценивание и улучшение в один шаг. Вместо точной оценки  $V^\pi$  для каждой политики сразу применяем оптимальный оператор Беллмана:

$$V_{k+1}(s) = \max_a \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V_k(s') \right].$$

После сходимости извлекаем политику:

$$\pi(s) \in \arg \max_a \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V(s') \right].$$

Оператор

$$(T^*V)(s) = \max_a \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V(s') \right]$$

тоже является  $\gamma$ -сжатием, поэтому  $V_k \rightarrow V^*$ .

## Сравнение Policy Iteration и Value Iteration

Policy Iteration обычно делает меньше внешних итераций, но каждая итерация дороже, потому что требует полноценного policy evaluation. Value Iteration делает более дешевые итерации, но их может быть больше. Оба метода требуют знания модели среды:  $p(s' | s, a)$  и  $r(s, a)$ , а также конечного или эффективно перебираемого пространства состояний и действий. Поэтому они относятся к динамическому программированию и model-based постановке.

## Что обязательно сказать на экзамене

Уравнения Беллмана - это рекурсивная декомпозиция “текущая награда плюс дисконтированная будущая ценность”. Для фиксированной политики получаем линейное уравнение  $V^\pi = r^\pi + \gamma P^\pi V^\pi$ . Для оптимальной политики появляется максимум по действиям. Policy Iteration чередует оценку и улучшение политики; Value Iteration сразу применяет оптимальный оператор Беллмана.

### 3 Табличные методы: Монте-Карло и Q-learning

#### Переход к model-free RL

В динамическом программировании предполагалось, что известны  $p(s' | s, a)$  и  $r(s, a)$ . В model-free RL эти величины неизвестны. Агент может только взаимодействовать со средой и получать переходы

$$(s_t, a_t, r_t, s_{t+1}).$$

Если  $|\mathcal{S}|$  и  $|\mathcal{A}|$  конечны и не слишком велики, можно хранить таблицу значений  $V(s)$  или  $Q(s, a)$ .

#### Общий вид стохастического обновления

Многие табличные методы имеют вид экспоненциального сглаживания:

$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha_k (y_k - Q_k(s, a)),$$

где  $y_k$  - целевое значение. Если  $y_k$  является шумной оценкой правильного таргета, то такое обновление является стохастической аппроксимацией. Для сходимости обычно нужны условия Роббинса-Монро:

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \quad \sum_{k=1}^{\infty} \alpha_k^2 < \infty.$$

Также нужно достаточно полное исследование среды: каждая пара  $(s, a)$  должна посещаться бесконечно часто.

#### Метод Монте-Карло

Методы Монте-Карло оценивают ценность по полным эпизодам. Возврат из момента  $t$ :

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k.$$

Для политики  $\pi$ :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a].$$

МС-оценка строится как среднее возвратов после посещения пары  $(s, a)$ :

$$Q(s, a) \approx \frac{1}{N(s, a)} \sum_{i: (s, a) \text{ visited}} G_i.$$

Онлайн-обновление:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha (G_t - Q(s_t, a_t)).$$

#### First-visit и every-visit МС

Есть две стандартные версии:

- first-visit МС - в одном эпизоде обновляем  $(s, a)$  только по первому посещению;
- every-visit МС - обновляем по каждому посещению.

Обе версии могут сходиться к  $Q^\pi$  при достаточном числе эпизодов.

#### Плюсы и минусы МС

Плюсы:

- не требует знания модели среды;
- дает несмещенную оценку полного возврата;
- хорошо распространяет информацию о награде вдоль всей траектории.

Минусы:

- нужно ждать конца эпизода;

- плохо подходит для бесконечных задач;
- высокая дисперсия, особенно при длинных эпизодах;
- данные используются не очень эффективно.

## Temporal Difference

TD-методы не ждут конца эпизода, а используют бутстраппинг:

$$V(s_t) \leftarrow V(s_t) + \alpha (r_t + \gamma V(s_{t+1}) - V(s_t)).$$

TD-ошибка:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

TD имеет меньшую дисперсию, чем MC, но появляется смещение, потому что таргет использует текущую оценку  $V$ .

## SARSA

SARSA - on-policy TD-метод для  $Q$ -функции. Название связано с пятеркой

$$(s_t, a_t, r_t, s_{t+1}, a_{t+1}).$$

Обновление:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha (r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)),$$

где  $a_{t+1}$  реально выбрано текущей behavior policy. Поэтому SARSA учит ценность той политики, которой агент фактически следует, например  $\varepsilon$ -жадной политики.

## Q-learning

Q-learning - off-policy TD-метод для обучения оптимальной  $Q^*$ -функции. Таргет использует жадное действие в следующем состоянии:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

Обновление:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left( r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right).$$

Соответствующая оптимальная политика:

$$\pi(s) \in \arg \max_a Q(s, a).$$

Почему off-policy: действие  $a_t$  может быть выбрано исследующей политикой, например  $\varepsilon$ -greedy, но таргет строится так, как будто дальше агент будет действовать жадно. То есть behavior policy и target policy различаются.

## $\varepsilon$ -greedy exploration

Для исследования среды часто используют  $\varepsilon$ -жадную стратегию:

$$a_t = \begin{cases} \text{случайное действие,} & \text{с вероятностью } \varepsilon, \\ \arg \max_a Q(s_t, a), & \text{с вероятностью } 1 - \varepsilon. \end{cases}$$

При убывающем  $\varepsilon$  агент сначала исследует, а затем постепенно переходит к эксплуатации найденной политики. Для теоретической сходимости важно, чтобы все пары  $(s, a)$  продолжали посещаться достаточно часто.

## Сходимость Q-learning

В табличном случае Q-learning сходится к  $Q^*$  при условиях:

- конечные  $\mathcal{S}$  и  $\mathcal{A}$ ;
- все пары  $(s, a)$  посещаются бесконечно часто;
- шаги обучения удовлетворяют условиям Роббинса-Монро;
- награды ограничены;
- $\gamma < 1$  для дисконтированной задачи.

## MC, SARSA, Q-learning: сравнение

| Метод       | Таргет                           | Тип              | Особенности                                         |
|-------------|----------------------------------|------------------|-----------------------------------------------------|
| Monte-Carlo | $G_t$                            | обычно on-policy | несмещенный, высокая дисперсия, нужен конец эпизода |
| SARSA       | $r + \gamma Q(s', a')$           | on-policy        | учитывает реальное исследующее поведение            |
| Q-learning  | $r + \gamma \max_{a'} Q(s', a')$ | off-policy       | учит жадную оптимальную политику                    |

## Что обязательно сказать на экзамене

Монте-Карло оценивает  $Q^\pi$  по полным возвратам и поэтому имеет низкое смещение, но высокую дисперсию. TD и Q-learning используют бутстраппинг: таргет содержит текущую оценку будущей ценности. Q-learning учит  $Q^*$  через таргет  $r + \gamma \max_{a'} Q(s', a')$  и является off-policy методом.

## 4 DQN и модификации: Double DQN, PER, dueling, noisy, multi-step, memory

### Переход от Q-learning к DQN

Табличный Q-learning неприменим, когда состояний слишком много или они непрерывные, например изображения Atari. Идея DQN: заменить таблицу  $Q(s, a)$  нейросетью  $Q_\theta(s, a)$ .

Если действий мало и они дискретны, удобная архитектура принимает состояние  $s$  и выдает вектор:

$$Q_\theta(s, \cdot) = (Q_\theta(s, a_1), \dots, Q_\theta(s, a_{|A|})) .$$

Тогда жадное действие:

$$a_t = \arg \max_a Q_\theta(s_t, a).$$

### Базовый DQN

Для перехода

$$T = (s, a, r, s', done)$$

строится таргет:

$$y(T) = r + \gamma(1 - done) \max_{a'} Q_{\theta^-}(s', a'),$$

где  $\theta^-$  - параметры target network. Лосс:

$$L(\theta) = \mathbb{E}_{T \sim \mathcal{D}} (Q_\theta(s, a) - y(T))^2 .$$

Алгоритм DQN:

1. Инициализировать основную сеть  $Q_\theta$ , target-сеть  $Q_{\theta^-}$  и replay buffer  $\mathcal{D}$ .
2. На каждом шаге выбрать действие через  $\varepsilon$ -greedy по  $Q_\theta$ .
3. Сохранить переход  $(s, a, r, s', done)$  в буфер.
4. Сэмплировать мини-батч переходов из  $\mathcal{D}$ .
5. Построить таргеты  $y(T)$  через target network.
6. Сделать шаг градиентного спуска по MSE/Huber loss.
7. Периодически обновлять target network:  $\theta^- \leftarrow \theta$ .

Две ключевые стабилизации DQN:

- replay buffer разрушает корреляции между соседними переходами и позволяет переиспользовать опыт;
- target network фиксирует таргеты на несколько шагов и уменьшает эффект “бежим за движущейся целью”.

### Double DQN

В vanilla DQN возникает overestimation bias: максимум по шумным оценкам склонен переоценивать истинное значение:

$$\mathbb{E} \max_a \hat{Q}(s, a) \geq \max_a Q(s, a).$$

Double DQN разделяет выбор действия и оценку действия. Выбор делается основной сетью, оценка - target-сетью:

$$a^*(s') = \arg \max_{a'} Q_\theta(s', a'),$$

$$y^{DDQN} = r + \gamma(1 - done) Q_{\theta^-}(s', a^*(s')).$$

Идея: одна сеть выбирает  $\arg \max$ , другая оценивает его значение. Это снижает переоценку.

## Prioritized Experience Replay

В обычном replay buffer переходы сэмпляются равномерно. Но не все переходы одинаково полезны. В PER вероятность выбора перехода зависит от TD-ошибки:

$$\delta_i = y_i - Q_\theta(s_i, a_i),$$
$$p_i = |\delta_i| + \varepsilon, \quad P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}.$$

Параметр  $\alpha$  управляет силой приоритизации:  $\alpha = 0$  дает равномерный replay.

Такой сэмплинг вносит смещение, поэтому используют importance sampling weights:

$$w_i = \left( \frac{1}{N} \frac{1}{P(i)} \right)^\beta,$$

обычно нормируют на максимум в батче. Итоговый лосс:

$$L(\theta) = \mathbb{E}_i \left[ w_i (y_i - Q_\theta(s_i, a_i))^2 \right].$$

Параметр  $\beta$  постепенно увеличивают к 1, чтобы к концу обучения компенсировать смещение сильнее.

## Dueling architecture

Dueling DQN разлагает  $Q$  на ценность состояния и преимущество действия:

$$Q(s, a) = V(s) + A(s, a).$$

Но такое разложение неоднозначно: можно прибавить константу к  $V$  и вычесть из  $A$ . Поэтому используют нормировку:

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A_\theta(s, a').$$

Интуиция: в некоторых состояниях важно знать, что состояние хорошее само по себе, даже если действия почти одинаковы. Dueling-сеть лучше учит  $V(s)$  и полезна, когда много действий имеют похожий эффект.

## Noisy Networks

Вместо внешнего  $\varepsilon$ -greedy можно добавить параметрический шум в веса сети. В noisy linear layer:

$$W = \mu_W + \sigma_W \odot \epsilon_W, \quad b = \mu_b + \sigma_b \odot \epsilon_b,$$

где  $\epsilon$  - шум, а  $\mu, \sigma$  обучаются. Тогда политика случайна из-за случайных параметров сети. Это дает более согласованное исследование: агент не просто случайно дергает действие на каждом шаге, а следует некоторой случайно возмущенной стратегии.

Преимущество над  $\varepsilon$ -greedy: exploration становится зависящим от состояния и обучаемым.

## Multi-step DQN

Вместо одношагового таргета можно использовать  $n$ -шаговый возврат:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n (1 - done) \max_{a'} Q_{\theta^-}(s_{t+n}, a').$$

Плюсы:

- награда быстрее распространяется назад;
- уменьшается зависимость от неточной bootstrap-оценки на один шаг.

Минусы:

- выше дисперсия;
- сложнее использовать off-policy данные, потому что несколько действий внутри возврата могли быть сделаны старой behavior policy.

## Память и частичная наблюдаемость

Обычный DQN предполагает, что состояние  $s_t$  марковское. В Atari часто используют frame stacking: состояние составляется из последних кадров, чтобы агент мог восстановить скорость объектов. В общем POMDP текущее наблюдение  $o_t$  не содержит всей информации о состоянии. Тогда используют память:

$$h_t = f_\theta(h_{t-1}, o_t, a_{t-1}), \quad Q_\theta(h_t, a).$$

Практически это DRQN/DQN с LSTM или GRU. Из replay buffer тогда сэмпляют не отдельные переходы, а последовательности, чтобы корректно обучать recurrent state.

## Rainbow DQN как комбинация улучшений

Многие модификации совместимы. Rainbow DQN объединяет несколько идей:

- Double DQN;
- prioritized replay;
- dueling architecture;
- multi-step targets;
- distributional RL;
- noisy nets.

Главная мысль: современный DQN - это не один трюк, а набор стабилизаций и улучшений сэмпловой эффективности.

## Что обязательно сказать на экзамене

DQN - это Q-learning с нейросетью, replay buffer и target network. Double DQN борется с overestimation bias. PER чаще показывает сети переходы с большой TD-ошибкой. Dueling architecture отдельно учит ценность состояния и advantage действий. Noisy Nets дают обучаемое исследование. Multi-step targets ускоряют распространение награды. Memory/recurrent DQN нужна для частично наблюдаемых сред.

## 5 Distributional RL: QR-DQN и Implicit Quantile Networks

### Мотивация

Обычный RL оценивает математическое ожидание суммарной награды:

$$Q^\pi(s, a) = \mathbb{E}[Z^\pi(s, a)],$$

где случайная величина

$$Z^\pi(s, a) = \sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a$$

задает весь reward-to-go. Distributional RL предлагает учесть не только среднее, а распределение  $Z^\pi(s, a)$ .

Причина: разные действия могут иметь одинаковое среднее, но разные риски. Одно действие стабильно дает средний результат, другое иногда дает огромный выигрыш, иногда провал. Обычный  $Q$  видит только среднее, а distributional-подход хранит больше информации о неопределенности исходов.

### Distributional Bellman Equation

Для фиксированной политики:

$$Z^\pi(s, a) \stackrel{D}{=} r(s, a) + \gamma Z^\pi(S', A'),$$

где

$$S' \sim p(\cdot \mid s, a), \quad A' \sim \pi(\cdot \mid S').$$

Символ  $\stackrel{D}{=}$  означает равенство по распределению. Это не равенство чисел, а равенство законов случайных величин.

Для оптимального управления:

$$Z^*(s, a) \stackrel{D}{=} r(s, a) + \gamma Z^*(S', a^*), \quad a^* \in \arg \max_{a'} \mathbb{E}[Z^*(S', a')].$$

Действие все равно обычно выбирается по среднему:

$$a^*(s) = \arg \max_a \mathbb{E}[Z(s, a)].$$

### Метрика Вассерштейна

Для распределений естественно использовать расстояние Вассерштейна. В одномерном случае его удобно выражать через квантильные функции:

$$W_p(U, V) = \left( \int_0^1 |F_U^{-1}(\tau) - F_V^{-1}(\tau)|^p d\tau \right)^{1/p}.$$

Distributional Bellman operator является сжимающим в максимальной форме метрики Вассерштейна. Это важная теоретическая мотивация distributional RL.

### QR-DQN: представление распределения квантилями

QR-DQN аппроксимирует распределение  $Z(s, a)$  набором  $N$  квантильных значений:

$$Z_\theta(s, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(s, a)},$$

где  $\theta_i(s, a)$  - положение  $i$ -го атома/квантиля. Уровни квантилей фиксированы:

$$\tau_i = \frac{i - 0.5}{N}, \quad i = 1, \dots, N.$$

Сеть для каждого действия выдает  $N$  чисел. Среднее для выбора действия:

$$Q(s, a) = \frac{1}{N} \sum_{i=1}^N \theta_i(s, a).$$

## Target QR-DQN

Для перехода  $(s, a, r, s', done)$  выбираем следующее действие по среднему:

$$a^* = \arg \max_{a'} \frac{1}{N} \sum_{j=1}^N \theta_j^-(s', a').$$

Таргетные квантили:

$$y_j = r + \gamma(1 - done)\theta_j^-(s', a^*), \quad j = 1, \dots, N.$$

Теперь надо приблизить набор предсказанных квантилей  $\theta_i(s, a)$  к набору таргетных квантилей  $y_j$ .

## Quantile regression loss

Для квантиля уровня  $\tau$  используется pinball loss:

$$\rho_\tau(u) = u(\tau - \mathbf{1}\{u < 0\}).$$

Для устойчивости применяют Huber-версию:

$$\rho_\tau^\kappa(u) = |\tau - \mathbf{1}\{u < 0\}| \mathcal{L}_\kappa(u),$$

где

$$\mathcal{L}_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa(|u| - \frac{1}{2}\kappa), & |u| > \kappa. \end{cases}$$

Ошибка между предсказанными и таргетными квантилями:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^\kappa(y_j - \theta_i(s, a)).$$

## Алгоритм QR-DQN

1. Сеть выдает квантили  $\theta_i(s, a)$  для каждого действия.
2. Действие выбирается  $\varepsilon$ -жадно по среднему квантилей.
3. Переход сохраняется в replay buffer.
4. Для батча переходов строятся distributional targets.
5. Сеть обучается quantile regression loss.
6. Target network обновляется как в DQN.

QR-DQN отличается от DQN не тем, как выбирается действие, а тем, что вместо одного числа  $Q(s, a)$  учится распределение будущего возврата.

## Implicit Quantile Networks

В QR-DQN уровни квантилей фиксированы. IQN делает сильнее: сеть принимает не только состояние, но и уровень квантиля  $\tau \in (0, 1)$ :

$$Z_\theta(s, a; \tau) \approx F_{Z(s, a)}^{-1}(\tau).$$

То есть IQN аппроксимирует всю квантильную функцию, а не только фиксированный набор точек.

Обычно  $\tau$  кодируется признаками, например через cosine embedding:

$$\phi(\tau) = \text{ReLU}(W[\cos(\pi i \tau)]_{i=1}^n + b),$$

после чего embedding состояния перемножается или объединяется с embedding квантиля. Сеть выдает значение квантиля для каждого действия.

## Таргет и лосс IQN

Сэмплируем уровни квантилей  $\tau_i$  для текущей сети и  $\tau'_j$  для таргета:

$$\tau_i, \tau'_j \sim U(0, 1).$$

Выбор действия:

$$a^* = \arg \max_{a'} \frac{1}{N'} \sum_{j=1}^{N'} Z_{\theta^-}(s', a'; \tau'_j).$$

Таргеты:

$$y_j = r + \gamma(1 - done) Z_{\theta^-}(s', a^*; \tau'_j).$$

Лосс:

$$L(\theta) = \frac{1}{NN'} \sum_{i=1}^N \sum_{j=1}^{N'} \rho_{\tau_i}^{\kappa} (y_j - Z_{\theta}(s, a; \tau_i)).$$

## Почему IQN лучше QR-DQN

QR-DQN хранит фиксированную дискретизацию распределения. IQN может сэмплировать любые уровни квантилей и тем самым аппроксимирует распределение гибче. Это дает более точную оценку хвостов распределения и лучше масштабируется по числу квантильных точек: можно менять количество сэмплов  $\tau$  на обучении и инференсе.

## Risk-sensitive политики

Distributional RL позволяет выбирать действия не только по среднему. Например:

- risk-averse: оптимизировать нижние квантили;
- risk-seeking: оптимизировать верхние квантили;
- CVaR: среднее по худшему хвосту распределения.

Но в базовых QR-DQN/IQN действие обычно выбирают по математическому ожиданию.

## Что обязательно сказать на экзамене

Distributional RL учит распределение возврата  $Z(s, a)$ , а не только его среднее  $Q(s, a)$ . Основное уравнение:  $Z(s, a) \stackrel{D}{=} r + \gamma Z(s', a')$ . QR-DQN представляет распределение фиксированным набором квантилей и обучает их quantile regression loss. IQN принимает уровень квантиля  $\tau$  на вход и аппроксимирует всю квантильную функцию.

## 6 Внутренняя мотивация: RND и ICM

### Проблема exploration

В задачах с разреженной наградой агент долго не получает полезный сигнал. Например, награда +1 выдается только при достижении цели, а во всех остальных состояниях 0 или маленький штраф. Тогда случайное исследование может почти никогда не привести к цели.

Идея внутренней мотивации: добавить к внешней награде  $r^{extr}$  дополнительную внутреннюю награду  $r^{intr}$ , которая поощряет исследование:

$$r(s, a) = r^{extr}(s, a) + \alpha r^{intr}(s, a),$$

где  $\alpha$  - коэффициент масштаба.

Для фиксированной политики суммы ценностей раскладываются:

$$Q^\pi(s, a) = Q_{extr}^\pi(s, a) + \alpha Q_{intr}^\pi(s, a),$$

$$V^\pi(s) = V_{extr}^\pi(s) + \alpha V_{intr}^\pi(s).$$

Но для оптимальных  $Q^*, V^*$  такое разложение в общем случае неверно, потому что максимум суммы не равен сумме максимумов.

### Простейшие count-based бонусы

В табличных средах можно поощрять новые состояния:

$$r^{intr}(s) = \frac{1}{\sqrt{N(s)}}$$

или похожей убывающей функцией от числа посещений  $N(s)$ . Чем реже состояние посещалось, тем больше бонус.

В больших пространствах состояний точные счетчики не работают. Поэтому используют приближения: хэширование, density models, prediction error. RND и ICM относятся к prediction-based подходам.

### Random Network Distillation

RND оценивает новизну состояния через ошибку предсказания случайной фиксированной сети.

Есть две сети:

- target network  $f(s)$  - случайно инициализированная и замороженная;
- predictor network  $\hat{f}_\theta(s)$  - обучаемая сеть, которая пытается предсказывать выход target network.

Внутренняя награда:

$$r_t^{intr} = \left\| \hat{f}_\theta(s_t) - f(s_t) \right\|^2.$$

Если состояние часто встречалось, predictor уже научился предсказывать  $f(s)$ , ошибка мала. Если состояние новое, ошибка велика, агент получает бонус.

Лосс обучения predictor:

$$L_{RND}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left\| \hat{f}_\theta(s) - f(s) \right\|^2.$$

### Почему target network случайная

Случайная сеть  $f$  задает фиксированное случайное embedding-пространство. Агент не должен оптимизировать саму задачу предсказания target network через изменение target network: цель должна быть неподвижной. Поэтому обучается только predictor.

Такой подход похож на дистилляцию: predictor пытается воспроизвести выходы случайного учителя.

## Преимущества и проблемы RND

Преимущества:

- простая реализация;
- работает в высокоразмерных наблюдениях;
- не требует модели действий;
- хорошо подходит для sparse reward.

Проблемы:

- ошибка предсказания зависит от масштаба признаков, нужна нормализация;
- внутреннюю награду надо нормировать;
- стохастический шум в наблюдениях может выглядеть как новизна;
- при долгом обучении бонусы исчезают, и агент может перестать исследовать.

## Intrinsic Curiosity Module

ICM строит внутреннюю награду через ошибку предсказания следующего состояния в learned feature space. Основная идея: агенту интересно то, что он плохо предсказывает, но что зависит от его действий.

ICM содержит:

- энкодер  $\phi(s)$ , переводящий состояние в признаки;
- inverse dynamics model: по  $\phi(s_t)$  и  $\phi(s_{t+1})$  предсказывает действие  $a_t$ ;
- forward dynamics model: по  $\phi(s_t)$  и  $a_t$  предсказывает  $\phi(s_{t+1})$ .

Inverse model:

$$\hat{a}_t = g_\eta(\phi(s_t), \phi(s_{t+1})).$$

Лосс inverse model для дискретных действий обычно cross-entropy:

$$L_{inv} = -\log p_\eta(a_t \mid \phi(s_t), \phi(s_{t+1})).$$

Forward model:

$$\widehat{\phi(s_{t+1})} = F_\psi(\phi(s_t), a_t).$$

Лосс forward model:

$$L_{fwd} = \frac{1}{2} \|F_\psi(\phi(s_t), a_t) - \phi(s_{t+1})\|^2.$$

Внутренняя награда:

$$r_t^{intr} = \frac{\eta}{2} \|F_\psi(\phi(s_t), a_t) - \phi(s_{t+1})\|^2.$$

## Зачем нужна inverse model

Если просто предсказывать следующий кадр, агент может получать большую внутреннюю награду за неуправляемый шум: мерцание телевизора, случайные текстуры, другие агенты. Inverse model заставляет энкодер  $\phi$  сохранять информацию, связанную с действием агента. Если по двум соседним признакам нельзя восстановить действие, такие признаки не очень полезны для управляемой динамики.

Иными словами, inverse model помогает игнорировать неуправляемые аспекты наблюдения, а forward model измеряет ошибку предсказания в пространстве управляемых признаков.

## Общий лосс ICM

Обычно оптимизируют комбинацию:

$$L_{ICM} = (1 - \beta)L_{inv} + \beta L_{fwd},$$

а агент максимизирует награду

$$r_t = r_t^{extr} + \alpha r_t^{intr}.$$

Параметр  $\beta$  балансирует обучение признаков через inverse и forward задачи.

## **RND против ICM**

RND измеряет новизну состояния: “я раньше не видел такое состояние”. ICM измеряет непредсказуемость перехода, зависящего от действия: “я не понимаю, что произойдет после моего действия”. RND проще, но хуже отделяет управляемую новизну от случайного шума. ICM сложнее, но через inverse dynamics пытается фокусироваться на аспектах среды, контролируемых агентом.

### **Что обязательно сказать на экзамене**

Внутренняя мотивация добавляет к внешней награде бонус за исследование. RND дает бонус как ошибку предсказания случайной замороженной сети. ICM дает бонус как ошибку forward model в feature space, а inverse model обучает признаки быть связанными с действиями агента. Главная цель - помочь exploration в sparse reward задачах.

## 7 Policy Gradient. REINFORCE и модификации

### Зачем нужны policy gradient методы

Value-based методы учат  $Q(s, a)$  и выбирают действие через  $\arg \max_a Q(s, a)$ . Это удобно для дискретных действий, но сложно для непрерывных: максимум по действиям может быть дорогим. Policy gradient методы напрямую параметризуют политику:

$$\pi_\theta(a \mid s)$$

и оптимизируют ожидаемую награду:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right].$$

### Likelihood-ratio trick

Пусть вероятность траектории:

$$p_\theta(\tau) = p(s_0) \prod_{t \geq 0} \pi_\theta(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t).$$

Динамика среды не зависит от  $\theta$ , поэтому

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Используем трюк:

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau).$$

Тогда

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log p_\theta(\tau) R(\tau)].$$

### Policy Gradient Theorem

Классическая форма:

$$\nabla_\theta J(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a)],$$

где дисконтированное распределение посещения состояний:

$$d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid \pi).$$

Эквивалентная траекторная форма:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^{\pi_\theta}(s_t, a_t).$$

### REINFORCE

В REINFORCE неизвестное  $Q^\pi(s_t, a_t)$  заменяется Монте-Карло возвратом:

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k.$$

Оценка градиента:

$$\widehat{\nabla_\theta J} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i} \gamma^t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) G_t^i.$$

Параметры обновляют градиентным подъемом:

$$\theta \leftarrow \theta + \alpha \widehat{\nabla_{\theta} J}.$$

В терминах loss для градиентного спуска часто пишут:

$$L(\theta) = - \sum_t \log \pi_{\theta}(a_t | s_t) G_t.$$

## Интуиция REINFORCE

Если действие привело к большому будущему возврату  $G_t$ , то увеличиваем  $\log \pi_{\theta}(a_t | s_t)$ , то есть повышаем вероятность этого действия. Если возврат малый или отрицательный, вероятность действия уменьшается. Это прямой метод “усиливать действия, которые встретились в хороших траекториях”.

## Проблема дисперсии

REINFORCE имеет несмещенную, но очень шумную оценку градиента:

- нужно ждать завершения эпизода;
  - возврат  $G_t$  может иметь большую дисперсию;
  - одно действие получает кредит за весь будущий эпизод, включая случайность среды.
- Поэтому чистый REINFORCE редко используется в сложных задачах без модификаций.

## Baseline

Можно вычитать из  $Q$  любую функцию состояния  $b(s)$ :

$$\nabla_{\theta} J(\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s,a} [\nabla_{\theta} \log \pi_{\theta}(a | s) (Q^{\pi}(s, a) - b(s))].$$

Это не меняет матожидание градиента, потому что

$$\mathbb{E}_{a \sim \pi} \nabla_{\theta} \log \pi_{\theta}(a | s) b(s) = b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) = 0.$$

На практике выбирают

$$b(s) = V^{\pi}(s).$$

Тогда появляется advantage:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

## REINFORCE with baseline

Оценка градиента:

$$\widehat{\nabla_{\theta} J} = \sum_t \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t)).$$

Если  $b(s) = V_{\phi}(s)$ , то критик обучается регрессией:

$$L_V(\phi) = \sum_t (V_{\phi}(s_t) - G_t)^2.$$

## Reward-to-go вместо полного возврата

Вместо полного возврата всей траектории  $R(\tau)$  для каждого действия используют только будущий возврат  $G_t$ . Это снижает дисперсию: действие  $a_t$  не должно получать кредит за награды, случившиеся до него.

## Entropy regularization

Чтобы политика не схлопывалась слишком рано, добавляют бонус энтропии:

$$J_{ent}(\theta) = J(\theta) + \beta \mathbb{E}_{s \sim d^\pi} H(\pi_\theta(\cdot | s)),$$

где

$$H(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi} \log \pi(a | s).$$

Это поощряет exploration и делает распределение действий менее вырожденным.

## On-policy характер

Policy gradient в базовом виде on-policy: данные должны быть собраны текущей политикой  $\pi_\theta$ . Если данные собраны старой политикой, оценка градиента для новой политики становится смещенной. Позднее это исправляют через importance sampling, TRPO/PPO и actor-critic схемы.

## Что обязательно сказать на экзамене

Policy Gradient напрямую оптимизирует параметры политики. Основная формула:  $\nabla J = \mathbb{E}[\nabla \log \pi(a | s) Q(s, a)]$ . REINFORCE заменяет  $Q$  на MC-return  $G_t$ . Главная проблема - высокая дисперсия. Baseline  $V(s)$  не меняет матожидание градиента, но снижает дисперсию, приводя к advantage  $A = Q - V$ .

## 8 Actor-Critic. A2C и GAE

### Идея Actor-Critic

Actor-Critic объединяет policy gradient и value learning. Есть две модели:

- actor  $\pi_\theta(a | s)$  - политика, выбирающая действия;
- critic  $V_\phi(s)$  или  $Q_\phi(s, a)$  - оценочная функция, оценивающая действия или состояния.

Actor обновляется по policy gradient, critic обучается предсказывать ценность и используется для уменьшения дисперсии.

Базовая формула градиента:

$$\nabla_\theta J(\theta) = \mathbb{E} [\nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t)].$$

Actor-Critic заменяет истинный  $A^\pi$  оценкой критика.

### TD-оценка advantage

Если critic оценивает  $V_\phi(s)$ , то одношаговая TD-ошибка:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

Она является оценкой advantage:

$$\delta_t \approx A^\pi(s_t, a_t).$$

Тогда actor update:

$$\nabla_\theta J \approx \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t.$$

Critic обучается по таргету

$$y_t = r_t + \gamma V_\phi(s_{t+1}),$$

лосс:

$$L_V(\phi) = \sum_t (V_\phi(s_t) - y_t)^2.$$

На практике в таргете градиент через  $V_\phi(s_{t+1})$  останавливают.

### A2C

A2C - Advantage Actor-Critic. Это синхронная версия A3C: несколько параллельных окружений собирают rollout, затем один общий батч используется для обновления actor и critic.

Общий loss часто записывают так:

$$L(\theta, \phi) = L_{policy} + c_v L_{value} - c_e L_{entropy},$$

где

$$L_{policy}(\theta) = - \sum_t \log \pi_\theta(a_t | s_t) \hat{A}_t,$$

$$L_{value}(\phi) = \sum_t (V_\phi(s_t) - \hat{R}_t)^2,$$

$$L_{entropy} = \sum_t H(\pi_\theta(\cdot | s_t)).$$

Знак перед entropy отрицательный в loss, потому что при минимизации мы хотим максимизировать энтропию.

## **$n$ -step returns в A2C**

Вместо одношагового TD можно использовать  $n$ -шаговый таргет:

$$\hat{R}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V_\phi(s_{t+n}).$$

Advantage:

$$\hat{A}_t^{(n)} = \hat{R}_t^{(n)} - V_\phi(s_t).$$

При  $n = 1$  получаем TD(0), при  $n \rightarrow \infty$  - Monte-Carlo return без bootstrap. Это уже показывает bias-variance trade-off.

## **Bias-variance trade-off**

Одношаговый TD:

- низкая дисперсия;
- высокий bias, потому что сильно зависит от текущего критика.

Monte-Carlo:

- низкий bias;
- высокая дисперсия.

$n$ -step return находится между ними. Нужно выбрать  $n$ , но фиксированный  $n$  не всегда оптимален.

## **GAE: Generalized Advantage Estimation**

GAE объединяет разные  $n$ -step оценки advantage с экспоненциальными весами. Сначала вводим TD residual:

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t).$$

GAE:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V.$$

Параметр  $\lambda \in [0, 1]$  управляет компромиссом:

- $\lambda = 0$ : одношаговый TD advantage  $\hat{A}_t = \delta_t$ ;
- $\lambda = 1$ : оценка близка к Monte-Carlo advantage;
- промежуточные значения дают баланс bias и variance.

## **Связь GAE с $n$ -step advantage**

$n$ -step advantage:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}) - V(s_t).$$

Его можно представить как сумму TD-ошибок:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}^V.$$

GAE - это экспоненциально взвешенная смесь таких оценок:

$$\hat{A}_t^{GAE} = (1 - \lambda) \left( \hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots \right).$$

Эквивалентная компактная форма - сумма TD residuals с весами  $(\gamma \lambda)^l$ .

## Почему Actor-Critic может быть on-policy и off-policy

Классический policy gradient требует on-policy данные. Но critic часто можно обучать off-policy, потому что Bellman targets используют переходы из буфера. Поэтому actor-critic - это семейство методов. A2C/PPO обычно on-policy. DDPG/TD3/SAC - off-policy actor-critic: critic учится по replay buffer, actor улучшается через critic.

## Преимущества Actor-Critic

По сравнению с REINFORCE:

- ниже дисперсия за счет критика;
- можно обновляться до конца эпизода;
- лучше сэмпловая эффективность;
- естественная работа с непрерывными действиями через actor.

Недостаток: появляется bias из-за неточного критика. Если critic плохой, actor получает неправильный сигнал.

## Что обязательно сказать на экзамене

Actor-Critic использует actor для политики и critic для оценки advantage. A2C оптимизирует policy loss, value loss и entropy bonus. Advantage можно оценивать через TD residual,  $n$ -step return или GAE. GAE задается формулой  $\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}$  и управляет bias-variance компромиссом через  $\lambda$ .

## 9 TRPO: Trust Region Policy Optimization

### Проблема больших шагов policy gradient

Обычный policy gradient делает шаг по параметрам политики:

$$\theta_{k+1} = \theta_k + \alpha \widehat{\nabla J(\theta_k)}.$$

Но маленькое изменение параметров не всегда означает маленькое изменение политики. Если политика изменилась слишком сильно, собранные старой политикой данные становятся нерелевантны, а качество может резко упасть. TRPO решает это через ограничение на изменение политики.

### Relative Performance Identity

Для двух политик  $\pi$  и  $\pi_{old}$  выполняется связь:

$$J(\pi) - J(\pi_{old}) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^\pi, a \sim \pi} [A^{\pi_{old}}(s, a)],$$

где

$$A^{\pi_{old}}(s, a) = Q^{\pi_{old}}(s, a) - V^{\pi_{old}}(s).$$

Это точная формула, но неудобная: матожидание берется по  $d^\pi$  - распределению состояний новой политики, которого у нас еще нет.

### Суррогатный функционал

TRPO заменяет распределение состояний новой политики на старое:

$$L_{\pi_{old}}(\pi) = J(\pi_{old}) + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_{old}}, a \sim \pi} [A^{\pi_{old}}(s, a)].$$

Так как действия в данных собраны из  $\pi_{old}$ , применяют importance sampling:

$$\mathbb{E}_{a \sim \pi} [A(s, a)] = \mathbb{E}_{a \sim \pi_{old}} \left[ \frac{\pi(a | s)}{\pi_{old}(a | s)} A(s, a) \right].$$

Итого практический surrogate:

$$L_{\pi_{old}}(\theta) = \mathbb{E}_{s, a \sim \pi_{old}} \left[ \frac{\pi_\theta(a | s)}{\pi_{old}(a | s)} A^{\pi_{old}}(s, a) \right].$$

Обозначение ratio:

$$\rho_\theta(s, a) = \frac{\pi_\theta(a | s)}{\pi_{old}(a | s)}.$$

### Trust region

Чтобы суррогат был надежным, новая политика не должна сильно отличаться от старой. TRPO решает задачу:

$$\max_{\theta} \mathbb{E}_{s, a \sim \pi_{old}} [\rho_\theta(s, a) A^{\pi_{old}}(s, a)]$$

при ограничении

$$\mathbb{E}_{s \sim d^{\pi_{old}}} [\text{KL}(\pi_{old}(\cdot | s) \| \pi_\theta(\cdot | s))] \leq \delta.$$

$\delta$  - радиус trust region. Он ограничивает среднее KL-расстояние между старой и новой политикой.

### Теоретическая гарантия

Основная идея теории TRPO: можно получить нижнюю оценку на качество новой политики через суррогатный функционал и штраф за отличие политик. В грубом виде:

$$J(\pi) \geq L_{\pi_{old}}(\pi) - C \cdot D_{KL}^{max}(\pi_{old}, \pi),$$

где  $C$  зависит от максимального advantage и  $\gamma$ . Поэтому если максимизировать surrogate и не уходить далеко по KL, можно ожидать монотонного улучшения.

Практически TRPO заменяет сложное ограничение на максимальный KL средним KL и решает приближенную constrained optimization задачу.

## Квадратичная аппроксимация ограничения

Пусть  $g = \nabla_{\theta} L(\theta)|_{\theta=\theta_{old}}$ . KL около старых параметров аппроксимируется через матрицу Фишера  $F$ :

$$\text{KL}(\theta_{old}, \theta_{old} + x) \approx \frac{1}{2} x^{\top} F x.$$

Тогда локальная задача:

$$\max_x g^{\top} x \quad \text{s.t.} \quad \frac{1}{2} x^{\top} F x \leq \delta.$$

Решение направлено вдоль natural gradient:

$$x^* \propto F^{-1} g.$$

С учетом ограничения:

$$x^* = \sqrt{\frac{2\delta}{g^{\top} F^{-1} g}} F^{-1} g.$$

## Natural gradient

Обычный градиент измеряет шаг в евклидовом пространстве параметров. Natural gradient учитывает геометрию распределений политик:

$$\tilde{\nabla} J = F^{-1} \nabla J.$$

Он делает шаг, который более осмыслен в пространстве вероятностных распределений, а не в пространстве параметров нейросети.

## Практический алгоритм TRPO

1. Собрать trajectories текущей политикой  $\pi_{old}$ .
2. Оценить  $A^{\pi_{old}}(s, a)$ , например через GAE.
3. Построить surrogate objective

$$L(\theta) = \mathbb{E} \left[ \rho_{\theta}(s, a) \hat{A}(s, a) \right].$$

4. Посчитать градиент  $g = \nabla_{\theta} L$ .
5. Приблизительно решить  $Fx = g$  через conjugate gradient, не формируя  $F$  явно.
6. Выбрать длину шага, удовлетворяющую KL-ограничению.
7. Сделать line search: принять шаг, если surrogate вырос и KL не превышает  $\delta$ .
8. Обновить critic отдельно.

## Плюсы и минусы TRPO

Плюсы:

- стабильнее обычного policy gradient;
- теоретически мотивирует монотонное улучшение;
- контролирует размер обновления политики.

Минусы:

- сложная реализация;
- нужен conjugate gradient и line search;
- метод on-policy и требует много новых данных;
- дороже, чем PPO.

## Что обязательно сказать на экзамене

TRPO максимизирует surrogate  $\mathbb{E}[\frac{\pi_{\theta}}{\pi_{old}} A_{old}]$ , но ограничивает средний KL между новой и старой политикой. Это trust region: новая политика не должна уходить далеко, чтобы surrogate оставался надежной оценкой улучшения. Локально TRPO приводит к natural gradient с ограничением  $\frac{1}{2} x^{\top} F x \leq \delta$ .

## 10 Bias-variance, GAE и PPO

### Bias-variance trade-off в RL

В actor-critic методах нужно оценивать advantage. Есть два крайних варианта.

Monte-Carlo:

$$\hat{A}_t^{MC} = \sum_{l=0}^{T-t} \gamma^l r_{t+l} - V(s_t).$$

Он почти не имеет bias, если траектория полная и  $V$  не используется в таргете, но имеет большую дисперсию.

TD(0):

$$\hat{A}_t^{TD} = r_t + \gamma V(s_{t+1}) - V(s_t) = \delta_t.$$

Он имеет меньшую дисперсию, но больший bias из-за bootstrap через неточный  $V$ .

$n$ -step оценка:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}) - V(s_t).$$

Чем больше  $n$ , тем меньше bias и больше variance. Чем меньше  $n$ , тем больше зависимость от критика и ниже variance.

### GAE

Generalized Advantage Estimation вводит TD residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

И определяет:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}.$$

Параметр  $\lambda$ :

- $\lambda = 0$ : только одношаговая TD-ошибка;
- $\lambda = 1$ : почти Monte-Carlo advantage;
- $0 < \lambda < 1$ : компромисс.

На практике часто используют  $\lambda \approx 0.95$ .

Возврат для обучения critic:

$$\hat{R}_t = \hat{A}_t + V(s_t).$$

### Переход от TRPO к PPO

TRPO стабилен, но сложен. PPO сохраняет идею “не менять политику слишком сильно”, но заменяет constrained optimization на простой loss, который можно оптимизировать обычным SGD/Adam.

Ratio новой и старой политики:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{old}(a_t | s_t)}.$$

Суррогат без ограничения:

$$L^{PG}(\theta) = \mathbb{E}_t[\rho_t(\theta) \hat{A}_t].$$

Если  $\rho_t$  сильно отличается от 1, политика слишком сильно изменила вероятность действия.

## РРО с клиппированием

Основной clipped objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[ \min \left( \rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

Здесь  $\epsilon$  обычно 0.1–0.3.

Интуиция по знаку advantage:

- Если  $\hat{A}_t > 0$ , действие было лучше ожидаемого. Мы хотим увеличить его вероятность, но не более чем в  $1 + \epsilon$  раз.
- Если  $\hat{A}_t < 0$ , действие было хуже ожидаемого. Мы хотим уменьшить его вероятность, но не слишком сильно: не ниже  $1 - \epsilon$ .

Клиппирование обнуляет стимул делать слишком большой шаг в направлении, которое уже достаточно изменило политику.

## Полный РРО loss

Обычно РРО оптимизирует:

$$L(\theta, \phi) = -L^{CLIP}(\theta) + c_1 L^{VF}(\phi) - c_2 L^{ENT}(\theta),$$

где

$$L^{VF}(\phi) = \mathbb{E}_t \left( V_\phi(s_t) - \hat{R}_t \right)^2,$$

$$L^{ENT}(\theta) = \mathbb{E}_t [H(\pi_\theta(\cdot | s_t))].$$

При минимизации знак перед  $L^{CLIP}$  отрицательный, потому что его надо максимизировать.

## Клиппирование критика

В некоторых реализациях клиппируют value update:

$$V_\phi^{clip}(s_t) = V_{old}(s_t) + \text{clip}(V_\phi(s_t) - V_{old}(s_t), -\epsilon_v, \epsilon_v).$$

Value loss:

$$L^{VF} = \max \left( (V_\phi(s_t) - \hat{R}_t)^2, (V_\phi^{clip}(s_t) - \hat{R}_t)^2 \right).$$

Цель та же: не позволять критику слишком резко менять оценки.

## РРО с KL penalty

Есть альтернативная версия РРО:

$$L^{KL PEN}(\theta) = \mathbb{E}_t [\rho_t(\theta) \hat{A}_t] - \beta \mathbb{E}_t [\text{KL}(\pi_{old}(\cdot | s_t) \| \pi_\theta(\cdot | s_t))].$$

Можно адаптировать  $\beta$ : если KL слишком большой, увеличить штраф; если слишком маленький, уменьшить. Но clipped РРО обычно проще и популярнее.

## Алгоритм РРО

1. Собрать rollout текущей политикой  $\pi_{old}$ .
2. Оценить значения  $V(s_t)$  и advantage  $\hat{A}_t$ , обычно через GAE.
3. Посчитать returns  $\hat{R}_t = \hat{A}_t + V(s_t)$ .
4. Зафиксировать log-prob старой политики  $\log \pi_{old}(a_t | s_t)$ .
5. Несколько эпох оптимизировать clipped objective на этом же батче.
6. Обновить  $\pi_{old} \leftarrow \pi_\theta$  и собрать новый rollout.

РРО остается on-policy: один rollout можно переиспользовать несколько эпох, но нельзя слишком долго обучаться на старых данных, потому что ratio и KL станут ненадежными.

## Почему PPO стал популярным

PPO проще TRPO: не нужны conjugate gradient и line search. При этом он сохраняет ключевую идею ограничения шага политики. Метод достаточно стабилен, хорошо работает в дискретных и непрерывных действиях, легко реализуется и стал стандартным baseline для on-policy deep RL.

## Что обязательно сказать на экзамене

Bias-variance trade-off возникает при оценке advantage: MC имеет низкий bias и высокую variance, TD - наоборот. GAE задает сглаженную сумму TD-ошибок  $\sum (\gamma\lambda)^l \delta_{t+l}$ . PPO заменяет сложный trust region из TRPO на clipped objective, который запрещает ratio  $\pi_\theta/\pi_{old}$  уходить далеко от 1 в выгодную для loss сторону.

## 11 DDPG и TD3: off-policy actor-critic для непрерывного управления

### Проблема непрерывных действий

DQN требует максимум по действиям:

$$\max_a Q(s, a).$$

В дискретном пространстве можно перебрать все действия. В непрерывном пространстве это дорого или невозможно. DDPG решает проблему, вводя детерминированного актора:

$$\mu_\theta(s) \approx \arg \max_a Q_\phi(s, a).$$

### Deterministic Policy Gradient

Для детерминированной политики  $a = \mu_\theta(s)$  градиент:

$$\nabla_\theta J(\mu_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^\mu} \left[ \nabla_\theta \mu_\theta(s) \nabla_a Q^\mu(s, a) \Big|_{a=\mu_\theta(s)} \right].$$

Интуиция: critic говорит, в какую сторону в пространстве действий надо сдвинуть действие, а актор через chain rule меняет параметры, чтобы выдавать более выгодные действия.

### DDPG

DDPG - off-policy actor-critic с replay buffer и target networks. Есть:

- actor  $\mu_\theta(s)$ ;
- critic  $Q_\phi(s, a)$ ;
- target actor  $\mu_{\theta^-}$ ;
- target critic  $Q_{\phi^-}$ .

Critic target:

$$y = r + \gamma(1 - \text{done})Q_{\phi^-}(s', \mu_{\theta^-}(s')).$$

Critic loss:

$$L_Q(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} (Q_\phi(s, a) - y)^2.$$

Actor objective:

$$J(\theta) = \mathbb{E}_{s \sim \mathcal{D}} Q_\phi(s, \mu_\theta(s)).$$

Actor update:

$$\nabla_\theta J \approx \mathbb{E}_{s \sim \mathcal{D}} \left[ \nabla_\theta \mu_\theta(s) \nabla_a Q_\phi(s, a) \Big|_{a=\mu_\theta(s)} \right].$$

Target networks обновляются мягко:

$$\phi^- \leftarrow \tau \phi + (1 - \tau) \phi^-, \quad \theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-.$$

### Exploration в DDPG

Поскольку политика детерминированная, для исследования добавляют шум:

$$a_t = \mu_\theta(s_t) + \epsilon_t.$$

Можно использовать гауссов шум

$$\epsilon_t \sim \mathcal{N}(0, \sigma^2 I),$$

или шум Орнштейна-Уленбека:

$$\epsilon_{t+1} = \alpha \epsilon_t + \mathcal{N}(0, \sigma^2 I),$$

который дает скоррелированные по времени возмущения. Это полезно в физических задачах, где случайное дерганье на каждом шаге неестественно.

## Проблемы DDPG

DDPG чувствителен к гиперпараметрам и ошибкам критика. Основные проблемы:

- overestimation bias в  $Q$ ;
- actor эксплуатирует ошибки критика;
- bootstrapping может разгонять ошибку;
- политика может стать нестабильной.

TD3 был предложен как набор исправлений DDPG.

## TD3: Twin Delayed DDPG

TD3 добавляет три ключевые идеи:

1. twin critics;
2. target policy smoothing;
3. delayed policy updates.

### Twin critics и clipped double Q

TD3 обучает два критика:

$$Q_{\phi_1}(s, a), \quad Q_{\phi_2}(s, a).$$

В таргете используется минимум:

$$y = r + \gamma(1 - done) \min_{i=1,2} Q_{\phi_i^-}(s', a').$$

Это уменьшает overestimation bias. Если один критик переоценил действие, минимум выбирает более осторожную оценку.

### Target policy smoothing

В таргетное действие добавляют сглаженный шум:

$$\begin{aligned} a' &= \mu_{\theta^-}(s') + \epsilon, \\ \epsilon &\sim \text{clip}(\mathcal{N}(0, \sigma^2), -c, c), \\ a' &\leftarrow \text{clip}(a', a_{low}, a_{high}). \end{aligned}$$

Таргет:

$$y = r + \gamma(1 - done) \min_i Q_{\phi_i^-}(s', a').$$

Интуиция: critic не должен давать слишком острые пики для отдельных действий. Сглаживание заставляет значение действия быть устойчивым к небольшим изменениям действия.

### Delayed policy updates

В TD3 actor обновляют реже, чем critic. Например, critic обновляется каждый gradient step, а actor и target networks - раз в  $d$  шагов. Это нужно, потому что actor должен обучаться по более надежному критику. Если actor обновлять слишком часто, он начинает эксплуатировать временные ошибки  $Q$ .

### Actor update в TD3

Actor использует только один из критиков, например  $Q_{\phi_1}$ :

$$\begin{aligned} J(\theta) &= \mathbb{E}_{s \sim \mathcal{D}} Q_{\phi_1}(s, \mu_{\theta}(s)), \\ \nabla_{\theta} J &= \mathbb{E}_{s \sim \mathcal{D}} [\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q_{\phi_1}(s, a)|_{a=\mu_{\theta}(s)}]. \end{aligned}$$

## DDPG и TD3: сравнение

| Компонент     | DDPG                   | TD3                            |
|---------------|------------------------|--------------------------------|
| Critic        | один $Q$               | два $Q$ , минимум в таргете    |
| Actor update  | каждый шаг             | с задержкой                    |
| Target action | $\mu^-(s')$            | $\mu^-(s')$ плюс clipped noise |
| Цель          | непрерывное управление | более стабильный DDPG          |

### Что обязательно сказать на экзамене

DDPG - это DQN-идея для непрерывных действий: вместо перебора  $\arg \max_a Q(s, a)$  вводится actor  $\mu(s)$ . Critic учится off-policy по replay buffer, actor обновляется через deterministic policy gradient. TD3 исправляет DDPG тремя трюками: два критика и минимум для борьбы с переоценкой, сглаживание таргетного действия и отложенные обновления actor.

## 12 Entropy RL и Soft Actor-Critic

### Maximum Entropy RL

В обычном RL максимизируется ожидаемая сумма наград:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \sum_{t \geq 0} \gamma^t r_t.$$

В Maximum Entropy RL добавляется энтропийный бонус:

$$J_{soft}(\pi) = \mathbb{E}_{\tau \sim \pi} \sum_{t \geq 0} \gamma^t [r_t + \alpha H(\pi(\cdot | s_t))],$$

где

$$H(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi} \log \pi(a | s).$$

Эквивалентно:

$$J_{soft}(\pi) = \mathbb{E}_{\tau \sim \pi} \sum_{t \geq 0} \gamma^t [r_t - \alpha \log \pi(a_t | s_t)].$$

$\alpha$  - temperature. Чем больше  $\alpha$ , тем сильнее поощрение случайности.

### Зачем нужна энтропия

Энтропийный бонус:

- улучшает exploration;
- мешает политике слишком рано схлопнуться в детерминированную;
- делает обучение устойчивее;
- позволяет получить стохастическую оптимальную политику.

Если награды в MDP всегда нулевые, оптимальная политика в max entropy смысле - политика максимальной энтропии, а не детерминированная.

### Soft value functions

Для простоты можно считать  $\alpha = 1$ ; в общем случае логарифм умножается на  $\alpha$ .

Soft Q-function:

$$Q_{soft}^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} V_{soft}^\pi(s').$$

Soft V-function:

$$V_{soft}^\pi(s) = \mathbb{E}_{a \sim \pi} [Q_{soft}^\pi(s, a) - \alpha \log \pi(a | s)].$$

Soft Bellman equation:

$$Q_{soft}^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s', a'} [Q_{soft}^\pi(s', a') - \alpha \log \pi(a' | s')].$$

### Soft policy improvement

Для фиксированного soft critic политика улучшается через задачу:

$$\max_{\pi} \mathbb{E}_{a \sim \pi(\cdot | s)} [Q(s, a) - \alpha \log \pi(a | s)].$$

Эквивалентно минимизации KL:

$$\pi_{new} = \arg \min_{\pi} \text{KL} \left( \pi(\cdot | s) \parallel \frac{\exp(Q(s, \cdot)/\alpha)}{Z(s)} \right).$$

Отсюда оптимальная soft-политика пропорциональна exponentiated Q:

$$\pi^*(a | s) \propto \exp(Q^*(s, a)/\alpha).$$

В непрерывных действиях нормировочная константа обычно не считается явно, поэтому политику аппроксимируют нейросетью.

## Soft Actor-Critic: общая идея

SAC - off-policy actor-critic для max entropy RL. Он использует:

- stochastic actor  $\pi_\theta(a \mid s)$ ;
- два soft Q-критика  $Q_{\phi_1}, Q_{\phi_2}$ ;
- target Q-networks;
- replay buffer;
- entropy term в actor и critic targets.

Современная версия SAC обычно не использует отдельную  $V_\psi$  сеть, хотя в ранней версии она была. В раннем варианте  $V_\psi(s)$  аппроксимировала

$$V(s) = \mathbb{E}_{a \sim \pi} [Q(s, a) - \alpha \log \pi(a \mid s)]$$

и использовалась для таргета критика.

## Critic update в SAC

Для перехода  $(s, a, r, s', done)$  сэмплируем

$$a' \sim \pi_\theta(\cdot \mid s').$$

Таргет:

$$y = r + \gamma(1 - done) \left[ \min_{i=1,2} Q_{\phi_i^-}(s', a') - \alpha \log \pi_\theta(a' \mid s') \right].$$

Critic loss:

$$L_Q(\phi_i) = \mathbb{E}_{\mathcal{D}} (Q_{\phi_i}(s, a) - y)^2, \quad i = 1, 2.$$

Минимум по двум критикам нужен, как в TD3, чтобы уменьшить overestimation bias.

## Actor update

Actor максимизирует soft value:

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta} \left[ \min_i Q_{\phi_i}(s, a) - \alpha \log \pi_\theta(a \mid s) \right].$$

Эквивалентный loss для минимизации:

$$L_\pi(\theta) = \mathbb{E}_{s \sim \mathcal{D}, \epsilon} \left[ \alpha \log \pi_\theta(a_\theta(s, \epsilon) \mid s) - \min_i Q_{\phi_i}(s, a_\theta(s, \epsilon)) \right].$$

Здесь используется reparameterization trick:

$$a_\theta(s, \epsilon) = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I).$$

## Автоматическая настройка температуры

Можно обучать  $\alpha$ , чтобы энтропия политики была близка к целевой  $\bar{H}$ :

$$L(\alpha) = \mathbb{E}_{a \sim \pi_\theta} [-\alpha(\log \pi_\theta(a \mid s) + \bar{H})].$$

Если энтропия слишком мала,  $\alpha$  растет и сильнее поощряет случайность. Если энтропия слишком велика,  $\alpha$  уменьшается.

## SAC как развитие Soft Q-Learning

Soft Q-Learning теоретически задает политику через

$$\pi(a \mid s) \propto \exp(Q(s, a)/\alpha).$$

Но в непрерывных действиях постоянно сэмплировать из такого распределения трудно. SAC добавляет actor, который явно аппроксимирует эту soft-greedy политику. Это похоже на идею DDPG: вместо поиска  $\arg\max$  по  $Q$  добавляем actor. Только в SAC actor стохастический и обучается с entropy regularization.

## Что обязательно сказать на экзамене

Maximum Entropy RL максимизирует не только награду, но и энтропию политики:  $r - \alpha \log \pi(a | s)$ . Soft Bellman equations содержат член  $-\alpha \log \pi$ . SAC - off-policy actor-critic: два Q-критика, стохастический actor, replay buffer, таргет  $r + \gamma(\min Q - \alpha \log \pi)$ . Actor оптимизирует  $\alpha \log \pi - Q$ , то есть балансирует высокую ценность и высокую энтропию.

## 13 Имитационное обучение, offline RL, Guided Cost Learning и GAIL

### Имитационное обучение

В RL часто трудно задать функцию награды. Иногда проще получить демонстрации эксперта:

$$\mathcal{D}_E = \{\tau_i\}, \quad \tau = (s_0, a_0, s_1, a_1, \dots).$$

Цель imitation learning - научить агента вести себя как эксперт.

### Behavioral Cloning

Behavioral Cloning сводит задачу к supervised learning:

$$\max_{\theta} \sum_{(s,a) \in \mathcal{D}_E} \log \pi_{\theta}(a | s).$$

Для дискретных действий это cross-entropy classification, для непрерывных - регрессия или максимизация likelihood гауссовой политики.

Проблема: covariate shift. Модель обучалась на состояниях эксперта, но при собственной ошибке попадает в состояния, которых не было в датасете. Ошибки накапливаются.

### Dagger

Dagger борется с covariate shift:

1. Обучить начальную политику на демонстрациях.
2. Запустить текущую политику и собрать посещенные состояния.
3. Попросить эксперта разметить правильные действия в этих состояниях.
4. Добавить данные в датасет и переобучить политику.

Минус: нужен эксперт, способный размечать новые состояния.

### Offline RL

Offline RL обучает политику по фиксированному датасету переходов:

$$\mathcal{D} = \{(s, a, r, s')\},$$

без новых взаимодействий со средой. Это важно, если реальные эксперименты дороги или опасны.

Главная проблема offline RL - distributional shift и extrapolation error. Если политика выбирает действия, которых почти нет в датасете, critic может сильно ошибаться:

$$Q(s, a) \quad \text{для out-of-distribution } a.$$

Затем actor начинает эксплуатировать эти ошибки.

Типичные идеи offline RL:

- ограничивать новую политику близостью к behavior policy;
- штрафовать OOD-действия;
- использовать conservative Q-learning;
- комбинировать RL objective с behavioral cloning loss.

### Inverse Reinforcement Learning

IRL пытается восстановить функцию награды, которую максимизирует эксперт. По демонстрациям нельзя однозначно восстановить награду: много разных reward functions могут объяснять одну и ту же политику. Поэтому нужны дополнительные предположения.

В Maximum Entropy IRL предполагают, что вероятность траектории эксперта пропорциональна экспоненте ее награды:

$$p_{\theta}(\tau) \propto \exp(R_{\theta}(\tau)) \prod_t p(s_{t+1} | s_t, a_t),$$

где

$$R_{\theta}(\tau) = \sum_t r_{\theta}(s_t, a_t).$$

Нормировочная константа:

$$Z(\theta) = \int \exp(R_{\theta}(\tau)) \prod_t p(s_{t+1} | s_t, a_t) d\tau.$$

## Guided Cost Learning

Guided Cost Learning обучает cost/reward model по экспертным траекториям. Лог-правдоподобие экспертной траектории:

$$\log p_{\theta}(\tau) = R_{\theta}(\tau) - \log Z(\theta) + \text{const}.$$

Градиент имеет вид разности признаков/наград на экспертных и сэмплированных траекториях:

$$\nabla_{\theta} \mathcal{L} = \mathbb{E}_{\tau \sim \mathcal{D}_E} \nabla_{\theta} R_{\theta}(\tau) - \mathbb{E}_{\tau \sim p_{\theta}} \nabla_{\theta} R_{\theta}(\tau).$$

Проблема: второе матожидание по текущей модели траекторий трудно считать. Поэтому используется сэмплирование траекторий текущей политики, которая обучается максимизировать найденную reward/cost function. Отсюда название guided: обучение reward и обучение policy направляют друг друга.

Общая схема:

1. Имеем экспертные траектории.
2. Обучаем reward/cost так, чтобы экспертные траектории были более вероятны, чем сгенерированные агентом.
3. Обучаем политику на текущей reward/cost.
4. Генерируем новые траектории агентом и повторяем.

## GAIL

GAIL - Generative Adversarial Imitation Learning. Он избегает явного восстановления reward function и формулирует imitation как GAN-подобную игру.

Есть:

- generator - политика  $\pi_{\theta}$ , порождающая траектории;
- discriminator  $D_w(s, a)$ , отличающий пары экспертных данных от пар агента.

Objective:

$$\min_{\pi} \max_D \mathbb{E}_{(s,a) \sim \pi_E} \log D(s, a) + \mathbb{E}_{(s,a) \sim \pi} \log(1 - D(s, a)) - \lambda H(\pi).$$

Discriminator обучается как бинарный классификатор: эксперт - 1, агент - 0. Политика обучается так, чтобы обмануть дискриминатор.

## Reward из дискриминатора

Для обновления политики используют reward, полученный из discriminator. Возможные варианты:

$$r(s, a) = -\log(1 - D(s, a))$$

или

$$r(s, a) = \log D(s, a) - \log(1 - D(s, a)).$$

Затем политика обновляется обычным RL-методом, например TRPO/PPO.

## Связь GAIL с occupancy measures

Осцирансу measure политики:

$$\rho_{\pi}(s, a) = \pi(a | s) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s | \pi).$$

GAIL можно понимать как минимизацию расстояния между осцирансу measure эксперта и агента. Если агент посещает те же пары  $(s, a)$  с теми же частотами, что эксперт, он имитирует поведение эксперта.

## GAN и GAIL: отличие

В GAN generator напрямую дифференцируемо преобразует шум в объект, и градиент от discriminator течет в generator. В GAIL generator - это политика внутри среды: действие влияет на будущие состояния через недифференцируемую динамику. Поэтому политика обучается не прямым backprop через discriminator, а RL-методом с reward от discriminator.

## Что обязательно сказать на экзамене

Behavioral cloning - supervised learning по демонстрациям, но страдает от covariate shift. Offline RL учится по фиксированному датасету и боится out-of-distribution действий. IRL восстанавливает награду из экспертных траекторий. Guided Cost Learning обучает reward через max entropy likelihood. GAIL заменяет явный reward adversarial-схемой: discriminator отличает эксперта от агента, а policy обучается обманывать discriminator через RL.

## 14 Многорукие бандиты: UCB и Thompson Sampling

### Постановка задачи

В задаче многорукого бандита есть набор действий/ручек  $a \in \mathcal{A}$ . На шаге  $k$  агент выбирает  $a_k$  и получает награду

$$r_k \sim p(r \mid a_k).$$

Ценность действия:

$$Q(a) = \mathbb{E}[r \mid a].$$

Оптимальная ценность:

$$V^* = \max_{a \in \mathcal{A}} Q(a).$$

Цель - минимизировать regret:

$$\mathcal{R}_T = TV^* - \sum_{k=0}^{T-1} Q(a_k).$$

Эквивалентно:

$$\mathcal{R}_T = \sum_{a \in \mathcal{A}} n_T(a)(V^* - Q(a)),$$

где  $n_T(a)$  - сколько раз выбрали действие  $a$ .

### Exploration-exploitation dilemma

Нужно балансировать:

- exploitation - выбирать действие с лучшей текущей оценкой;
- exploration - пробовать плохо изученные действия, которые могут оказаться лучше.

Жадный алгоритм может навсегда застрять на неоптимальной ручке из-за случайно удачного начального сэмпла.

### Оценка среднего

Эмпирическая оценка:

$$Q_k(a) = \frac{\sum_{i=0}^k r_i \mathbf{1}\{a_i = a\}}{\sum_{i=0}^k \mathbf{1}\{a_i = a\}}.$$

Инкрементальное обновление для выбранного действия:

$$Q_k(a_k) = Q_{k-1}(a_k) + \frac{1}{n_k(a_k)} (r_k - Q_{k-1}(a_k)).$$

Для нестационарных бандитов используют постоянный шаг:

$$Q_k(a_k) = Q_{k-1}(a_k) + \alpha(r_k - Q_{k-1}(a_k)).$$

### $\varepsilon$ -greedy

Простой подход:

$$a_k = \begin{cases} \text{случайное действие,} & \text{с вероятностью } \varepsilon, \\ \arg \max_a Q_k(a), & \text{с вероятностью } 1 - \varepsilon. \end{cases}$$

При постоянном  $\varepsilon$  regret обычно растет линейно, потому что агент продолжает с ненулевой вероятностью выбирать заведомо плохие действия.

## Lai-Robbins lower bound

Для хороших алгоритмов regret не может расти быстрее нижней логарифмической границы. В классической форме:

$$\mathbb{E}[\mathcal{R}_T] \geq \log T \sum_{a \neq a^*} \frac{V^* - Q(a)}{\text{KL}(p(\cdot | a) \| p(\cdot | a^*))}.$$

Алгоритм называется асимптотически оптимальным, если его regret растёт логарифмически.

## UCB: optimism in the face of uncertainty

UCB строит верхнюю доверительную границу для каждого действия:

$$a_k = \arg \max_a [Q_k(a) + U_k(a)].$$

Идея: выбирать действие с лучшим оптимистичным значением. Если действие мало пробовали,  $U_k(a)$  велик, и алгоритм его исследует. Если пробовали много, доверительный интервал сужается.

По неравенству Хёффдинга для наград в  $[0, 1]$ :

$$\mathbb{P}(\mu \geq \hat{\mu} + u) \leq e^{-2nu^2}.$$

Отсюда доверительный бонус:

$$U_k(a) = \sqrt{\frac{-\log \delta}{2n_k(a)}}.$$

В практическом UCB часто используют:

$$a_k = \arg \max_a \left[ Q_k(a) + c \sqrt{\frac{\log k}{n_k(a)}} \right].$$

## Алгоритм UCB

1. Каждое действие выбрать хотя бы один раз.
2. На шаге  $k$  выбрать

$$a_k = \arg \max_a \left[ Q_k(a) + c \sqrt{\frac{\log k}{n_k(a)}} \right].$$

3. Получить  $r_k$ .
4. Обновить  $Q_k(a_k)$  и  $n_k(a_k)$ .

UCB не требует случайного exploration-параметра  $\epsilon$ ; исследование возникает из неопределённости.

## Thompson Sampling

Thompson Sampling - байесовский probability matching. Для каждого действия есть неизвестный параметр  $\theta_a$  распределения наград. Поддерживаем posterior:

$$p(\theta_a | \text{data}).$$

На каждом шаге:

1. Для каждого действия сэмплировать

$$\tilde{\theta}_a \sim p(\theta_a | \text{data}).$$

2. Посчитать сэмплированную ценность

$$\tilde{Q}(a) = \mathbb{E}[r | \tilde{\theta}_a].$$

3. Выбрать

$$a_k = \arg \max_a \tilde{Q}(a).$$

4. Получить награду и обновить posterior выбранного действия.

Действие выбирается с вероятностью, равной posterior-вероятности быть оптимальным:

$$\pi(a) = \mathbb{P} \left( Q(a) = \max_{\hat{a}} Q(\hat{a}) \mid \text{data} \right).$$

### Bernoulli bandit и Beta posterior

Если награда  $r \in \{0, 1\}$ :

$$r \sim \text{Bernoulli}(\theta_a).$$

Сопряженный prior:

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a).$$

После наблюдения  $r$ :

$$\alpha_a \leftarrow \alpha_a + r, \quad \beta_a \leftarrow \beta_a + 1 - r.$$

Среднее posterior:

$$\mathbb{E}[\theta_a] = \frac{\alpha_a}{\alpha_a + \beta_a}.$$

Но Thompson Sampling выбирает не по среднему posterior, а сэмплирует  $\tilde{\theta}_a \sim \text{Beta}(\alpha_a, \beta_a)$  и выбирает максимум.

### UCB против Thompson Sampling

UCB использует оптимизм через верхнюю границу доверительного интервала. Thompson Sampling использует posterior uncertainty: если действие плохо изучено, его posterior широк, и оно иногда сэмплируется как лучшее. Оба метода естественно уменьшают exploration по мере накопления данных.

### Что обязательно сказать на экзамене

Бандит - MDP без состояния, где нужно минимизировать regret. UCB выбирает действие по  $Q_k(a) + c\sqrt{\log k / n_k(a)}$ , реализуя оптимизм перед неопределенностью. Thompson Sampling поддерживает posterior на параметры наград и выбирает действие, которое оказалось лучшим в сэмпле из posterior. Для Bernoulli-бандита Beta prior обновляется как  $\alpha \leftarrow \alpha + r$ ,  $\beta \leftarrow \beta + 1 - r$ .

## 15 Monte Carlo Tree Search

### Планирование

Планирование отличается от обучения политики тем, что для текущего состояния  $s_0$  мы ищем хороший план действий:

$$a_0, a_1, a_2, \dots$$

Если известна модель переходов и наград, задача:

$$\arg \max_{(a_0, a_1, \dots)} \mathbb{E} [R(\tau) \mid s_0, a_0, a_1, \dots].$$

В дискретных действиях можно строить дерево возможных продолжений. Но полное дерево растёт экспоненциально по глубине:

$$|\mathcal{A}|^H.$$

MCTS строит дерево выборочно, концентрируясь на перспективных ветвях.

### Дерево MDP

Корень дерева соответствует текущему состоянию  $s_0$ . Ребро соответствует действию  $a$ . Узел на глубине  $t$  соответствует плану  $(a_0, \dots, a_{t-1})$ . В узлах/ребрах хранят статистику:

- $n(N, a)$  - сколько раз проходили по ребру  $(N, a)$ ;
- $Q(N, a)$  - оценка ценности действия  $a$  из узла  $N$ ;
- $n(N) = \sum_a n(N, a)$  - число посещений узла.

### Четыре фазы MCTS

Одна итерация MCTS состоит из:

1. Selection - спуск по уже построенному дереву.
2. Expansion - добавление новых узлов.
3. Simulation/Evaluation - оценка новых листьев.
4. Backpropagation/Update - обновление статистик по пройденному пути.

### Selection

Selection выбирает путь от корня до листа по tree policy. Обычно используют UCB-подобное правило:

$$a = \arg \max_{a \in \mathcal{A}} \left[ Q(N, a) + C \sqrt{\frac{\log n(N)}{n(N, a)}} \right].$$

Первое слагаемое - exploitation: идти в ветви с высокой оценкой. Второе - exploration: пробовать редко посещенные ветви.

При спуске через ребро  $(N, a)$  симулятор генерирует следующий переход:

$$s', r \sim p(s', r \mid s, a).$$

### Expansion

Когда дошли до листа, дерево расширяют: для возможных действий создают новые дочерние узлы. Если действий слишком много, можно расширять не все действия, а случайный поднабор или progressive widening.

## Simulation / Evaluation

Нужно оценить ценность нового листа. В классическом MCTS запускают rollout до конца эпизода с default policy, часто случайной:

$$\pi_{default}(a \mid s).$$

Получают Monte-Carlo оценку возврата:

$$\hat{R} = \sum_{t=0}^T \gamma^t r_t.$$

В современных вариантах вместо случайного rollout можно использовать value network:

$$\hat{V}(s_{leaf}) \approx V(s_{leaf}).$$

## Backpropagation

После simulation полученную оценку возвращают назад по пройденному пути. Для каждого ребра  $(N, a)$ :

$$\begin{aligned} n(N, a) &\leftarrow n(N, a) + 1, \\ Q(N, a) &\leftarrow Q(N, a) + \frac{1}{n(N, a)} \left( \hat{R} - Q(N, a) \right). \end{aligned}$$

Если в пути были промежуточные награды, для каждого ребра используют соответствующий reward-to-go после этого ребра.

## Итоговая политика из MCTS

После  $K$  итераций MCTS выбирает действие в корне. Можно взять:

$$a_0 = \arg \max_a n(N_0, a)$$

или задать стохастическую политику по посещениям:

$$\pi(a \mid s_0) \propto n(N_0, a)^{1/T},$$

где  $T$  - температура. При  $T \rightarrow 0$  выбор становится жадным.

## Использование в реальной среде

Обычно MCTS работает online:

1. В текущем состоянии  $s_t$  сделать много итераций MCTS.
2. Выбрать реальное действие  $a_t$ .
3. Среда переходит в  $s_{t+1}$ .
4. Сохранить поддерево, соответствующее выбранному действию и новому состоянию.
5. Повторить планирование из нового корня.

Это похоже на receding horizon planning: постоянно перепланируем.

## MCTS и бандиты

В каждом узле MCTS нужно выбрать, какую ветвь исследовать. Это локальная задача многорукого бандита. Поэтому UCB естественно используется как tree policy. MCTS можно понимать как много вложенных бандитных задач, по одной в каждом узле дерева.

## Плюсы и минусы MCTS

Плюсы:

- не строит полное дерево;
- концентрируется на перспективных ветвях;
- anytime algorithm: чем больше итераций, тем лучше решение;
- хорошо работает в дискретных action spaces и играх.

Минусы:

- требует симулятор/модель;
- дорог по времени на каждый реальный action;
- сложен для непрерывных действий без модификаций;
- качество зависит от rollout policy/value heuristic.

## AlphaZero-style MCTS

В современных методах MCTS комбинируют с нейросетями:

- policy network дает prior  $P(s, a)$  для действий;
- value network оценивает лист вместо случайного rollout;
- tree policy использует PUCT:

$$a = \arg \max_a \left[ Q(s, a) + cP(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)} \right].$$

Это делает поиск намного эффективнее.

## Что обязательно сказать на экзамене

MCTS - метод планирования с симулятором для дискретных действий. Одна итерация: selection по UCB, expansion, simulation/evaluation, backpropagation. В ребрах хранят  $Q(N, a)$  и  $n(N, a)$ . После многих итераций действие выбирают по максимальному числу посещений или по распределению посещений.

## 16 LQR, iLQR и общая схема Model-based RL

### Model-based RL

Model-based RL явно или неявно обучает модель динамики среды:

$$p_{\theta}(s' | s, a)$$

или детерминированную модель

$$s_{t+1} = f_{\theta}(s_t, a_t).$$

Общая схема:

1. Собрать переходы  $(s, a, r, s')$  в датасет.
2. Обучить модель динамики и, если нужно, модель награды.
3. Использовать модель для планирования или генерации “сновидений”.
4. Улучшить политику.
5. Собрать новые данные и повторить.

Плюсы model-based RL:

- высокая sample efficiency;
- можно планировать без реального риска;
- можно использовать знания физики.

Минусы:

- модель ошибается;
- планировщик может эксплуатировать ошибки модели;
- ошибки накапливаются при длинных rollout.

### Планирование в непрерывном управлении

Если динамика и награда дифференцируемы:

$$s_{t+1} = f(s_t, a_t), \quad r_t = r(s_t, a_t),$$

можно оптимизировать последовательность действий:

$$\max_{a_{0:T-1}} \sum_{t=0}^{T-1} r(s_t, a_t)$$

при ограничениях динамики. Прямой способ - backpropagation through time по действиям. Но для систем управления есть классические методы: LQR и iLQR.

### Linear Quadratic Regulator

LQR решает задачу с линейной динамикой и квадратичной стоимостью. Обычно формулируется как минимизация cost:

$$x_{t+1} = Ax_t + Bu_t,$$
$$J = \sum_{t=0}^{T-1} (x_t^T Q x_t + u_t^T R u_t) + x_T^T Q_f x_T.$$

Здесь  $Q \succeq 0$ ,  $R \succ 0$ . Нужно найти управляющие воздействия  $u_t$ , минимизирующие  $J$ .

### Value function в LQR

Для LQR оптимальная value function квадратична:

$$V_t(x) = x^T P_t x.$$

На последнем шаге:

$$P_T = Q_f.$$

Рекурсия Риккати назад во времени:

$$P_t = Q + A^T P_{t+1} A - A^T P_{t+1} B (R + B^T P_{t+1} B)^{-1} B^T P_{t+1} A.$$

Оптимальное управление линейно по состоянию:

$$u_t = -K_t x_t,$$

где

$$K_t = (R + B^T P_{t+1} B)^{-1} B^T P_{t+1} A.$$

## Бесконечный горизонт

Для стационарной задачи бесконечного горизонта  $P_t \rightarrow P$ , где  $P$  решает алгебраическое уравнение Риккати:

$$P = Q + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A.$$

Тогда управление стационарно:

$$u = -Kx.$$

## Интуиция LQR

LQR балансирует две цели:

- держать состояние близко к нулю через штраф  $x^T Q x$ ;
- не тратить слишком большое управление через штраф  $u^T R u$ .

Если  $R$  большой, управление осторожное. Если  $Q$  большой, система агрессивнее возвращает состояние к цели.

## iLQR: iterative LQR

Реальные системы часто нелинейны:

$$x_{t+1} = f(x_t, u_t),$$

а cost не обязательно квадратичный:

$$J = \sum_t \ell(x_t, u_t) + \ell_f(x_T).$$

iLQR решает задачу итеративно: вокруг текущей номинальной траектории линеаризует динамику и квадратично аппроксимирует cost, затем решает локальную LQR-задачу.

## Линеаризация динамики

Пусть есть номинальная траектория  $(\bar{x}_t, \bar{u}_t)$ . Введем отклонения:

$$\delta x_t = x_t - \bar{x}_t, \quad \delta u_t = u_t - \bar{u}_t.$$

Динамика приближенно:

$$\delta x_{t+1} \approx f_x(t) \delta x_t + f_u(t) \delta u_t,$$

где

$$f_x(t) = \left. \frac{\partial f}{\partial x} \right|_{\bar{x}_t, \bar{u}_t}, \quad f_u(t) = \left. \frac{\partial f}{\partial u} \right|_{\bar{x}_t, \bar{u}_t}.$$

## Квадратичная аппроксимация cost

Cost раскладывают до второго порядка:

$$\ell(x_t, u_t) \approx \ell_t + \ell_x^T \delta x_t + \ell_u^T \delta u_t + \frac{1}{2} \delta x_t^T \ell_{xx} \delta x_t + \delta u_t^T \ell_{ux} \delta x_t + \frac{1}{2} \delta u_t^T \ell_{uu} \delta u_t.$$

После этого локальная задача становится LQR-подобной.

## Backward pass iLQR

Определяют локальную Q-функцию для отклонений:

$$Q(\delta x, \delta u) = \ell(x, u) + V_{t+1}(f(x, u)).$$

Ее производные дают оптимальное локальное управление:

$$\delta u_t^* = k_t + K_t \delta x_t,$$

где

$$k_t = -Q_{uu}^{-1} Q_u, \quad K_t = -Q_{uu}^{-1} Q_{ux}.$$

Здесь  $k_t$  - feedforward correction,  $K_t$  - feedback gain.

## Forward pass iLQR

После backward pass выполняют rollout новой траектории:

$$u_t^{new} = \bar{u}_t + \alpha k_t + K_t(x_t^{new} - \bar{x}_t),$$

где  $\alpha \in (0, 1]$  выбирается line search. Если cost уменьшился, новая траектория принимается. Затем процесс повторяется.

## Алгоритм iLQR

1. Инициализировать последовательность управлений  $u_{0:T-1}$ .
2. Сделать forward rollout через настоящую динамику и получить номинальную траекторию.
3. Линеаризовать динамику и квадратично аппроксимировать cost вдоль траектории.
4. Backward pass: посчитать  $k_t, K_t$ .
5. Forward pass: применить новое управление с line search.
6. Повторять до сходимости.

## Связь iLQR с model-based RL

Если настоящая динамика неизвестна, можно обучить модель  $f_\theta(s, a)$  и применять iLQR внутри этой модели. Тогда получаем model-based RL pipeline:

1. собрали данные на реальной среде;
2. обучили дифференцируемую модель динамики;
3. планируем через iLQR/CEM/MPC;
4. исполняем первое действие;
5. добавляем новые данные и переобучаем модель.

## Шумная динамика

Если переход стохастический:

$$s_{t+1} = f(s_t, a_t) + \epsilon_t,$$

то планирование по среднему может быть недостаточно. Возможные подходы:

- оптимизировать ожидаемый cost;
- учитывать ковариацию шума;
- использовать ансамбли моделей для неопределенности;
- планировать робастно или риск-чувствительно;
- перепланировать на каждом шаге в MPC-стиле.

## Что обязательно сказать на экзамене

LQR решает линейно-квадратичную задачу: линейная динамика, квадратичный cost, оптимальное управление  $u = -Kx$ , матрицы находятся рекурсией Риккати. iLQR обобщает это на нелинейную динамику: вокруг текущей траектории линеаризует динамику, квадратично аппроксимирует cost, решает локальный LQR, делает forward rollout и повторяет. В model-based RL модель динамики можно обучить по данным и использовать LQR/iLQR/MPC для планирования.